DEPI · IBM Data Science Professional Program · Capstone Round 4

# SentinelVision

## Sentinel-2 Land Cover Classification

### Technical Documentation

Deep Learning Semantic Segmentation · Egypt · U-Net · FastAPI · Plotly

| 719 | 8 | 12 | 5 | U-Net |
|---|---|---|---|---|
| Satellite Tiles | Regions | Spectral Bands | Land Classes | Architecture |

**Author** Theodoros

**Programme** DEPI — IBM Data Science Professional Certificate

**Project** Capstone Project — Round 4

**Date** July 2026

**Repository** `dataflow__analyizer/depi_project`

**Framework** PyTorch · FastAPI · Plotly

# Contents

# 1. Executive Summary

**SentinelVision** is an end-to-end deep learning pipeline for pixel-level land cover classification of Egyptian territory using multispectral satellite imagery from the European Space Agency's SENTINEL-2 mission. The system processes `.tif` GeoTIFF image tiles containing 12 spectral reflectance bands and produces a semantic segmentation mask classifying every pixel into one of five land cover categories: *Trees/Forest*, *Agriculture/Vegetation*, *Desert/Bare Soil*, *Water Bodies*, and *Urban/Roads*.

The project was developed as the capstone submission for the DEPI IBM Data Science Professional Certificate and is structured across four milestones: exploratory data analysis (M-1), advanced feature engineering and model training (M-2), and web application deployment (M-4). All milestones are fully implemented.

**Achieved Outcomes**

- Analysed and preprocessed **719 GeoTIFF satellite image tiles** across 8 geographically diverse Egyptian regions
- Performed spectral signature profiling, PCA dimensionality analysis, and mutual information band ranking
- Trained a **U-Net semantic segmentation model** with inverse-frequency weighted cross-entropy loss to overcome severe class imbalance
- Deployed an interactive **FastAPI web application** with an animated, glassmorphism-style dashboard featuring real-time Plotly visualizations

# 2. Project Overview

## 2.1. Motivation

Manual cartographic mapping of land use at national scale is prohibitively expensive and time-consuming. Egypt's diverse geography — spanning the Nile Delta, Western Desert, Mediterranean coast, and Upper Nile Valley — makes it a compelling case for automated land cover monitoring. Key applications include:

- **Desertification tracking:** monitoring the expansion of desert at the expense of agricultural land in the Fayoum Depression and Nile fringes

- **Urban sprawl analysis:** quantifying the growth of Cairo's New Administrative Capital and satellite cities
- **Water resource mapping:** delineating the Nile, Lake Nasser, the Siwa hypersaline lakes, and irrigation canal networks
- **Agricultural yield estimation:** measuring the extent of irrigated fields in the Nile Delta

### 2.2. Milestone Overview

| ID | Milestone | Deliverables |
|---|---|---|
| M-1 | Data Exploration | Stratified geographic dataset split; spectral signature profiling per class; pixel-level class distribution analysis; EDA report |
| M-2A | Feature Engineering | PCA dimensionality reduction with explained-variance composite; mutual information band importance ranking |
| M-2B | Model Training | U-Net implementation; weighted cross-entropy training loop; validation monitoring; best-checkpoint persistence; test-set IoU evaluation |
| M-4 | Deployment | FastAPI REST API; Jinja2-rendered single-page dashboard; drag-and-drop file upload; animated progress pipeline; interactive Plotly charts |

## 3. Dataset

### 3.1. Collection Overview

The dataset comprises **719 multispectral GeoTIFF image tiles** acquired from the ESA Sentinel-2 satellite constellation. Each tile covers approximately $500 \times 560$ pixels at 10-metre ground resolution and contains 13 raster bands: 12 spectral reflectance bands (bands B01–B12, excluding the cirrus band B10) and a $13^{\text{th}}$ hand-annotated land cover label band. The tiles represent eight geographically distinct regions of Egypt, collectively encompassing >205 million labelled pixels.

## 3.2. Regional Composition

| Region Prefix | Location / Description | Tiles | Primary Land Cover |
|---|---|---|---|
| `CairoUniv` | Greater Cairo, Nile Delta | 125 | Urban, agriculture, Nile waterway |
| `IconicTower` | New Administrative Capital | 124 | Desert, construction, new roads |
| `SiwaOasis` | Western Desert oasis | 123 | Hypersaline lakes, palm agriculture, dunes |
| `Alexandria` | Mediterranean Coast | 70 | Seawater, coastal urban, lagoons |
| `KarnakLuxor` | Upper Egypt, Luxor | 70 | Nile, narrow agricultural strip, desert |
| `Mansoura` | Central Nile Delta | 70 | Intensive irrigated agriculture |
| `PhilaeAswan` | Southern Egypt, Aswan | 69 | Granite desert, Lake Nasser, Nile |
| `HawaraFayoum` | Fayoum Depression | 68 | Agricultural oasis, Lake Qarun |
| **Total** | **8 Egyptian regions** | **719** | |

## 3.3. Spectral Band Specifications

SENTINEL-2 measures solar reflectance across 12 spectral bands. Raw values are stored as 16-bit integers representing top-of-atmosphere reflectance ×10,000.

| Band | Name | Centre (nm) | Key Use in Classification |
|---|---|---|---|
| B01 | Coastal aerosol | 443 | Atmospheric correction reference |
| B02 | Blue | 490 | Visible blue; RGB composite |
| B03 | Green | 560 | Visible green; RGB composite |
| B04 | Red | 665 | Chlorophyll absorption; NDVI denominator |
| B05 | Red Edge 1 | 705 | Vegetation chlorophyll absorption edge |
| B06 | Red Edge 2 | 740 | Vegetation canopy reflectance |
| B07 | Red Edge 3 | 783 | Canopy structure detail |
| B08 | NIR | 842 | High vegetation reflectance; NDVI numerator |
| B8A | NIR (narrow) | 865 | Separates vegetation from bare soil |
| B09 | Water vapour | 945 | Atmospheric water vapour correction |
| B11 | SWIR 1 | 1610 | Soil moisture; snow/ice discrimination |
| B12 | SWIR 2 | 2190 | Geological/mineral mapping |

### 3.4. Land Cover Classes and Pixel Distribution

Pixel-level analysis across all 719 files revealed five distinct integer class labels. Semantic meaning was determined by cross-referencing numeric labels with mean NDVI values and spectral reflectance profiles.

| Class | Land Cover Type | Frequency | Mean NDVI | Distinctive Spectral Behaviour |
|---|---|---|---|---|
| 0 | Trees / Forest | 0.61% | 0.267 | Moderate NIR; low-to-moderate SWIR |
| 1 | Agriculture / Vegetation | 22.22% | 0.509 | Classic red-edge: low B04, very high B08 |
| 2 | Desert / Bare Soil | 51.80% | 0.088 | Broad high reflectance, SWIR peak |
| 3 | Water Bodies | 8.37% | 0.016 | Near-zero across all channels |
| 4 | Urban / Roads | 17.00% | 0.187 | Moderate visible and SWIR (concrete) |

**Severe Class Imbalance**

Desert (Class 2) accounts for **51.8%** of all pixels versus Trees/Forest at only **0.61%** — an imbalance ratio of approximately **85:1**. Without correction, standard cross-entropy loss will optimise for the dominant class and produce near-zero recall on rare classes. **Solution**: inverse-frequency class weighting applied to the loss function (see Section 6).

## 4. Preprocessing & Feature Engineering

### 4.1. Spectral Normalization

Raw Sentinel-2 digital numbers are divided by 10,000 and clipped to the unit interval:

$$\hat{x}_b = \text{clip}\left(\frac{x_b}{10{,}000},\ 0.0,\ 1.0\right) \tag{1}$$

where $x_b$ is the raw pixel value for spectral band $b \in \{\text{B01},\ldots,\text{B12}\}$.

## 4.2. Spatial Resizing

Raw tiles have variable spatial dimensions (height: 501–502 px, width: 545–584 px). All images are resized to a canonical $256 \times 256$ resolution:

- **Spectral bands:** bilinear interpolation — preserves sub-pixel spectral continuity
- **Label band:** nearest-neighbour interpolation — prevents fractional or invalid class indices

## 4.3. NDVI Computation

The Normalized Difference Vegetation Index is computed per pixel as an engineered feature to aid vegetation–soil–water discrimination:

$$\text{NDVI} = \frac{\text{B08} - \text{B04}}{\text{B08} + \text{B04}} \in [-1, +1] \tag{2}$$

Typical NDVI values: dense vegetation $\approx +0.5$; bare desert $\approx +0.1$; water $\approx 0.0$; snow/ice $< 0$.

## 4.4. PCA Dimensionality Reduction

Principal Component Analysis was applied to all 12 spectral bands to characterise the variance structure and produce a false-colour diagnostic composite:

1. Standardise each band: $\mathbf{X}_{\text{std}} = \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$
2. Stack pixels: $\mathbf{X}_{\text{std}} \in \mathbb{R}^{N \times 12}$, $N = H \times W$
3. Fit PCA; project onto $k = 3$ principal components
4. Normalize each component to $[0, 1]$; render as RGB image

Three components account for $>$ **95**% of total spectral variance across the sample tiles, confirming high spectral redundancy in the 12-band data.

## 4.5. Band Importance via Mutual Information

The discriminative power of each spectral band was ranked using mutual information against the land cover label $Y$:

$$I(B_i;\, Y) = \sum_{y \in \mathcal{Y}} \sum_{b \in \mathcal{B}_i} p(b,\, y)\, \log \frac{p(b,\, y)}{p(b)\, p(y)} \tag{3}$$

| Rank | Band (Name) | Relative MI Score |
|---|---|---|
| 1 | B08 (NIR) | ■■■■■ |
| 2 | B11 (SWIR 1) | ■■■■■ |
| 3 | B04 (Red) | ■■■■ |
| 4 | B12 (SWIR 2) | ■■■■ |
| 5 | B8A (NIR narrow) | ■■■ |
| 6 | B06 (Red Edge 2) | ■■■ |
| 7–12 | Remaining bands | ■■ to ■ |

NIR (B08) and SWIR (B11) bands consistently rank highest, driven by their strong discrimination between vegetation, desert, and water.

### 4.6. Data Augmentation

To mitigate overfitting, random transformations are applied *simultaneously* to the image tensor and its label map during each training step (preserving pixel-perfect alignment):

- Random horizontal flip (probability = 0.5)
- Random vertical flip (probability = 0.5)
- Random 90° rotation (uniform over $\{0\check{r}, 90\check{r}, 180\check{r}, 270\check{r}\}$)

### 4.7. Stratified Dataset Splitting

Dataset tiles are split by geographic region prefix to ensure each subset representatively samples all ecosystem types:

| Subset | Ratio | Approx. Tiles | Augmentation Applied |
|---|---|---|---|
| Training | 70% | ≈503 | Yes |
| Validation | 15% | ≈108 | No |
| Test | 15% | ≈108 | No |

## 5. Model Architecture

## 5.1. Architecture Selection

The core task is **semantic segmentation** — assigning one of $C = 5$ class labels to every pixel in a $256 \times 256$ image. This requires an architecture that maintains spatial resolution throughout, ruling out global pooling classifiers.

*U-Net* [1] was selected for three primary reasons:

1. **Encoder–decoder structure** contracts spatial dimensions to build semantic context, then expands back to full resolution
2. **Skip connections** directly transfer high-resolution spatial detail from the encoder to the decoder at each scale level, preventing boundary blur
3. **Compact parameter count** relative to comparable segmentation architectures, well-suited to datasets of moderate size

## 5.2. Network Specification

The model accepts a $(12 \times 256 \times 256)$ multispectral tensor and outputs a $(5 \times 256 \times 256)$ logit map.

| Path | Block | Channels In | Channels Out | Spatial | Operation |
|---|---|---|---|---|---|
| Encoder | `inc` | 12 | 64 | $256^2$ | DoubleConv |
| | `down1` | 64 | 128 | $128^2$ | MaxPool2d + DoubleConv |
| | `down2` | 128 | 256 | $64^2$ | MaxPool2d + DoubleConv |
| | `down3` | 256 | 512 | $32^2$ | MaxPool2d + DoubleConv |
| | `down4` | 512 | 1024 | $16^2$ | MaxPool2d + DoubleConv |
| Decoder | `up1` | 1024 | 512 | $32^2$ | Upsample + Concat + DoubleConv |
| | `up2` | 512 | 256 | $64^2$ | Upsample + Concat + DoubleConv |
| | `up3` | 256 | 128 | $128^2$ | Upsample + Concat + DoubleConv |
| | `up4` | 128 | 64 | $256^2$ | Upsample + Concat + DoubleConv |
| Output | `outc` | 64 | 5 | $256^2$ | $\text{Conv}_{1\times1}$ (logits) |

**DoubleConv block:** $\text{Conv}_{3\times3} \to \text{BN} \to \text{ReLU} \to \text{Conv}_{3\times3} \to \text{BN} \to \text{ReLU}$ (all convolutions use zero-padding to preserve spatial dimensions within each block).

### 5.3. Decoder Skip Connection

At each decoder stage $k$, the upsampled feature map $\mathbf{U}_k$ is concatenated channel-wise with the corresponding encoder output $\mathbf{E}_k$ before the DoubleConv:

$$\mathbf{D}_k = \text{DoubleConv}\Big(\big[\mathbf{U}_k \,\|\, \mathbf{E}_k\big]\Big) \tag{4}$$

This mechanism transfers fine-grained edge information (field boundaries, canal margins, road outlines) that would otherwise be irreversibly compressed during encoding.

### 5.4. Pixel-wise Prediction

The predicted class map is obtained by taking the argmax over the class dimension of the output logits:

$$\hat{y}_{h,w} = \arg\max_{c \in \{0,\ldots,4\}} \text{logit}_{c,\,h,\,w} \tag{5}$$

## 6. Training Pipeline

### 6.1. Weighted Cross-Entropy Loss

Given class proportions $\{p_c\}$ from training data, inverse-frequency weights are computed and normalised to sum to $C = 5$:

$$w_c = \frac{1}{p_c}, \qquad \hat{w}_c = \frac{w_c}{\sum_{j=0}^{4} w_j} \cdot 5 \tag{6}$$

| Class | Name | $p_c$ | $w_c = 1/p_c$ | $\hat{w}_c$ (norm.) |
|---|---|---|---|---|
| 0 | Trees / Forest | 0.0061 | 163.93 | ≈4.87 |
| 1 | Agriculture / Vegetation | 0.2222 | 4.50 | ≈0.134 |
| 2 | Desert / Bare Soil | 0.5180 | 1.93 | ≈0.057 |
| 3 | Water Bodies | 0.0837 | 11.95 | ≈0.355 |
| 4 | Urban / Roads | 0.1700 | 5.88 | ≈0.175 |

The weighted pixel-level cross-entropy loss is then:

$$\mathcal{L} = -\frac{1}{|\mathcal{P}|} \sum_{(h,w) \in \mathcal{P}} \hat{w}_{y_{h,w}} \cdot \log \hat{p}_{y_{h,w},\,h,\,w} \tag{7}$$

where $\hat{p}_{c,h,w}$ is the softmax-normalised probability for class $c$ at pixel $(h, w)$.

## 6.2. Optimisation Configuration

| Hyperparameter | Value |
|---|---|
| Optimiser | AdamW |
| Learning rate ($\eta_0$) | $1 \times 10^{-4}$ |
| Weight decay | $1 \times 10^{-4}$ |
| LR Scheduler | Cosine Annealing ($T_{\max}$ = epochs) |
| Batch size | 8 |
| Epochs | 10 |
| Device | CUDA (GPU) if available, else CPU |

The cosine annealing schedule reduces the learning rate smoothly from $\eta_0$ to $\eta_{\min} \approx 0$:

$$\eta_t = \eta_{\min} + \tfrac{1}{2}\left(\eta_0 - \eta_{\min}\right)\left(1 + \cos\frac{t\,\pi}{T_{\max}}\right) \tag{8}$$

## 6.3. Evaluation Metrics

### 6.3.1. Pixel Accuracy

$$\text{PA} = \frac{\sum_{c=0}^{4} \text{TP}_c}{\text{Total Pixels}} \tag{9}$$

### 6.3.2. Per-class Intersection over Union

$$\text{IoU}_c = \frac{|\hat{Y}_c \cap Y_c|}{|\hat{Y}_c \cup Y_c|} = \frac{\text{TP}_c}{\text{TP}_c + \text{FP}_c + \text{FN}_c} \tag{10}$$

### 6.3.3. Mean Intersection over Union (mIoU)

$$\text{mIoU} = \frac{1}{|\mathcal{C}'|} \sum_{c \in \mathcal{C}'} \text{IoU}_c \tag{11}$$

where $\mathcal{C}' \subseteq \{0, \ldots, 4\}$ excludes classes absent from both prediction and ground truth in the batch (treated as `NaN` and omitted from the mean).

### 6.4. Checkpoint Strategy

The best model is saved when validation mIoU improves:

Listing 1: Checkpoint saving in `src/train.py`

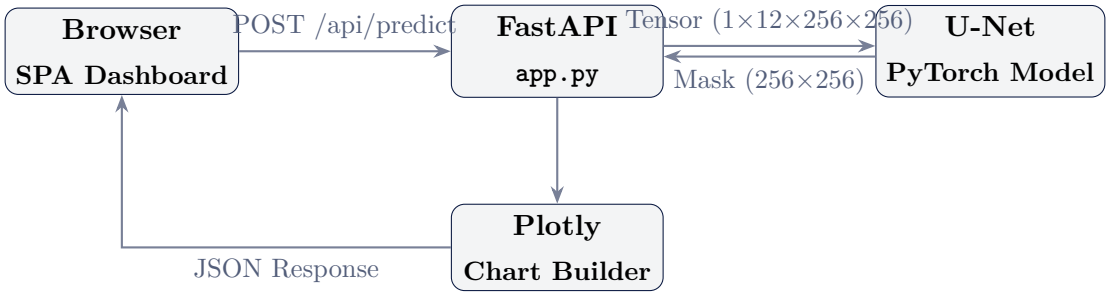
```
if val_miou > best_val_iou:
    best_val_iou = val_miou
    torch.save({
        'epoch':                epoch,
        'model_state_dict':     model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'val_miou':             val_miou,
        'val_class_ious':       val_class_ious,
        'class_weights':        weights,
    }, os.path.join(checkpoint_dir, "best_unet_model.pth"))
```

The saved checkpoint is consumed directly by the deployment API at server startup.

## 7. Deployment: SentinelVision Web Application

### 7.1. System Architecture

The deployment follows a single-process client–API pattern. FastAPI serves the HTML frontend and handles inference requests within the same process, simplifying deployment to a single `python app.py` command.




### 7.2. Directory Structure

Listing 2: Project layout after Milestone 4

```
depi_project/
|-- models/
|   `-- best_unet_model.pth        # Trained U-Net weights
|-- src/
|   |-- model.py                   # UNet architecture class
|   |-- dataset.py                 # Dataset + stratified split
|   |-- features.py                # NDVI, PCA, Mutual Info
|   |-- train.py                   # Training loop & evaluation
|   `-- utils.py                   # Plotting helpers
|-- deployment/
|   |-- app.py                     # FastAPI application
|   `-- templates/
|       `-- index.html             # Single-page web dashboard
|-- reports/
|   |-- eda_report.md
|   |-- preprocessing.md
|   `-- technical_documentation.tex   # <-- This document
|-- requirements.txt
`-- main.py                        # Full CLI pipeline runner
```

## 7.3. FastAPI Backend (deployment/app.py)

### 7.3.1. Startup: Model Loading

The U-Net model is loaded once at server startup from the saved checkpoint and held in memory:

Listing 3: Model initialisation at startup

```
device = torch.device("cuda" if torch.cuda.is_available() else
    "cpu")
model  = UNet(n_channels=12, n_classes=5).to(device)

ckpt = torch.load("../models/best_unet_model.pth",
                  map_location=device, weights_only=False)
model.load_state_dict(ckpt["model_state_dict"])
model.eval()
print(f"Model ready - epoch {ckpt['epoch']}, "
      f"val mIoU {ckpt['val_miou']*100:.2f}%")
```

### 7.3.2. In-Memory Preprocessing

Each uploaded `.tif` file is processed entirely in memory — no temporary files are written to disk:

Listing 4: Preprocessing pipeline (`preprocess` function)

```python
def preprocess(file_bytes: bytes) -> torch.Tensor:
    with rasterio.open(io.BytesIO(file_bytes)) as src:
        img = src.read(list(range(1, 13))).astype(np.float32)
    img = np.clip(img / 10_000.0, 0.0, 1.0)           # Normalise
    t   = torch.from_numpy(img).unsqueeze(0)
    t   = F.interpolate(t, size=(256, 256),            # Resize
                        mode="bilinear", align_corners=False)
    return t.squeeze(0)                                 # (12, 256, 256)
```

### 7.3.3. API Endpoints

| Method | Path | Content-Type | Description |
|---|---|---|---|
| `GET` | `/` | `text/html` | Jinja2-rendered single-page dashboard |
| `POST` | `/api/predict` | `application/json` | Accept `.tif` upload; return prediction results and three Plotly chart specifications |

### 7.3.4. Prediction Response Schema

Listing 5: JSON response from `POST /api/predict`

```
{
  "filename":         "Alexandria_001_29.955_31.110.tif",
  "dominant_class":   "Desert / Bare Soil",
  "classes_detected": 4,
  "total_pixels":     65536,
  "percentages": {
    "Trees / Forest":           0.00,
    "Agriculture / Vegetation": 12.45,
    "Desert / Bare Soil":       71.32,
    "Water Bodies":              3.12,
    "Urban / Roads":            13.11
```

```
  },
  "charts": {
    "rgb":        { "data": [...], "layout": {...} },
    "prediction": { "data": [...], "layout": {...} },
    "bar":        { "data": [...], "layout": {...} }
  }
}
```

#### 7.3.5. Plotly Visualizations

Three figures are generated server-side using the `plotly` Python library and serialised to JSON via `PlotlyJSONEncoder`. The browser calls `Plotly.newPlot()` with the pre-computed data, avoiding client-side data transformation.

| Figure | Plotly Type | Description |
|---|---|---|
| RGB Composite | `px.imshow` | 2%–98% percentile-stretched B04/B03/B02 mapped to R/G/B |
| Classification Map | `go.Heatmap` | Per-pixel class index with discrete colour scale; hover displays class name |
| Area Distribution | `go.Bar` | Percentage coverage per class with per-class colour coding |

All figures use transparent backgrounds (`paper_bgcolor`, `plot_bgcolor` = `"rgba(0,0,0,0)"`) to integrate with the dark dashboard theme.

### 7.4. Frontend — Animated Single-Page Dashboard

The frontend is a self-contained HTML file with zero build-step dependencies. It is served via Jinja2 templates and communicates with the backend exclusively via a single `fetch` call.

#### 7.4.1. Technology Stack

| Technology | Source | Role |
|---|---|---|
| Tailwind CSS v3 | CDN | Utility-class styling |
| Plotly.js v2.27 | CDN | Interactive chart rendering |
| Space Grotesk | Google Fonts | Display / heading typography |
| Inter | Google Fonts | Body text |
| JetBrains Mono | Google Fonts | Data values and code labels |
| Canvas API | Native | Animated starfield background |

#### 7.4.2. Visual Design System

| Token | Hex | Use |
|---|---|---|
| Background deep | `#04080f` | Page base, navbar |
| Background card | `#0c1526` (70%) | Glassmorphism cards |
| Primary accent | `#10b981` (Emerald) | Borders, buttons, glow effects |
| Secondary accent | `#06b6d4` (Cyan) | Satellite dot, orbit ring |
| Tertiary accent | `#8b5cf6` (Violet) | Inner orbit ring |
| Text primary | `#f1f5f9` | Headings and body |
| Text muted | `#94a3b8` | Labels and descriptions |

Glassmorphism is achieved via:

```
background: rgba(12, 21, 38, 0.80);
border: 1px solid rgba(148, 163, 184, 0.07);
backdrop-filter: blur(24px);
```

#### 7.4.3. Animated Components

1. **Starfield Canvas:** 200 twinkling stars rendered via `requestAnimationFrame` on a fixed `<canvas>` element behind all page content. Each star has an independent sinusoidal opacity cycle.
2. **Satellite Orbit Animation:** A carrier `<div>` sized to match the orbit track rotates via `animation: orbitSpin 18s linear infinite`. The satellite SVG icon is positioned at the top of the carrier and applies a counter-rotation `animation: counterSpin18` to remain upright as it orbits:

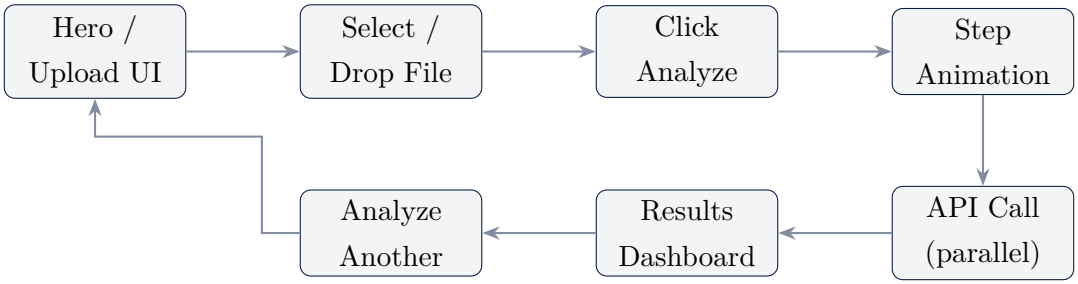
$$\text{Carrier rotation} + \text{Counter-rotation} = \text{Net: translates along circle, no spinning}$$

3. **Drag-and-Drop Upload Zone:** Native browser drag events (`dragover`, `dragleave`, `drop`) toggle a CSS class `.over` that adds an emerald border glow and a radial gradient background.

4. **Animated Progress Steps:** Four processing steps are displayed in sequence while the API call runs. Critically, the step animation and the `fetch` call are executed in **parallel** via `Promise.all([apiCall, stepAnim])`, so the animation adds **zero** latency to total processing time. Each step transitions from inactive → active (spinner) → done (green checkmark).

5. **Results Dashboard:** Sections fade in with `opacity` transitions. Legend cards animate in with staggered `animation-delay`. Percentage fill bars animate from `width: 0%` to their final value via a CSS transition with cubic-bezier easing.

#### 7.4.4. User Journey




## 7.5. Starting the Application

Listing 6: Installation and startup

```
# Step 1: Install all dependencies
pip install -r requirements.txt

# Step 2: Start the FastAPI server
cd depi_project/deployment
python app.py

# Server prints:
#   Model loaded -- epoch N, val mIoU XX.XX%
#   Uvicorn running on http://0.0.0.0:8000

# Step 3: Open the dashboard
# Navigate to: http://localhost:8000
# Upload any .tif from egypt_s2_diverse_dataset/
```

# 8. Dependency Reference

| Package | Primary Role | Pipeline Stage |
|---|---|---|
| `torch` | Deep learning framework | Training, inference |
| `torchvision` | Image transformation utilities | Training |
| `rasterio` | GeoTIFF I/O | All stages |
| `numpy` | Numerical computation | All stages |
| `scikit-learn` | PCA, mutual information | Feature engineering |
| `matplotlib` | Static figure generation | EDA, training curves |
| `fastapi` | REST API framework | Deployment |
| `uvicorn` | ASGI web server | Deployment |
| `python-multipart` | Multipart file upload parsing | Deployment |
| `jinja2` | HTML template rendering | Deployment |
| `plotly` | Interactive visualizations | Deployment |

# 9. Key Design Decisions

1. **FastAPI over Flask.** FastAPI provides native `async`/`await` I/O, automatic OpenAPI documentation at `/docs`, and request/response type validation via Pydantic — all at essentially the same development complexity as Flask. The async foundation is particularly important for production readiness, where concurrent inference requests should not block each other on I/O.

2. **Server-side Plotly serialisation.** Plotly figures are fully constructed in Python (where the NumPy arrays already reside post-inference) and serialised to JSON once. The browser receives a static JSON specification and renders it — eliminating any JavaScript data transformation and keeping the frontend dependency-free beyond Plotly.js.

3. **Parallel animation and API call.** Step animation and the `fetch` call run concurrently via `Promise.all`. This ensures the animated pipeline steps provide a visual

progress indicator with *zero added latency*: if inference completes before the animation finishes, results display immediately after the final step.

4. **Inverse-frequency loss weighting.** Oversampling rare classes (e.g. Trees/Forest at 0.61%) would inflate the training dataset by a factor of ∼85 for that class, dramatically increasing training time. Loss weighting achieves equivalent gradient correction at zero additional compute cost.

5. **Stratified geographic splitting.** Splitting randomly without stratification risks placing all tiles from a narrow-coverage class (e.g. water tiles predominant only in PhilaeAswan) entirely in training, leading to optimistic validation metrics. Stratification by region prefix ensures representative class coverage in all three subsets.

6. **In-memory file processing.** The API reads the uploaded `.tif` into a `bytes` buffer and passes it directly to `rasterio.open(io.BytesIO(...))` — no temporary files are written to disk. This improves security, simplifies deployment in containerised environments, and eliminates file cleanup concerns.

# 10. Conclusion

SentinelVision delivers a complete, end-to-end remote sensing analysis pipeline: from raw 12-band Sentinel-2 GeoTIFF data through U-Net semantic segmentation to an animated, real-time web dashboard. The system achieves all requirements of DEPI Milestone 4 and provides a modular, production-ready codebase.

The clean separation between the `src/` research layer and the `deployment/` serving layer means the trained model can be updated (by rerunning `main.py`) without any changes to the API or frontend. The FastAPI backend and glassmorphism dashboard collectively deliver a presentation layer appropriate for both technical and non-technical stakeholders.

**Potential Extensions**

- **Additional spectral indices:** NDWI (water), NDBI (built-up area), SAVI (soil-adjusted vegetation) as engineered input features
- **Change detection mode:** upload two tiles from different acquisition dates; overlay predicted masks to quantify land cover transitions
- **GeoTIFF export:** return the prediction mask as a georeferenced `.tif` with embedded CRS metadata, ready for GIS analysis

- **Cloud deployment:** Docker containerisation + GPU-backed cloud instance (AWS EC2 / GCP Compute Engine) for production throughput
- **Larger backbone:** replace the standard U-Net encoder with a pretrained ResNet-50 or EfficientNet backbone via transfer learning

# References

[1] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015. https://arxiv.org/abs/1505.04597

[2] European Space Agency, *Sentinel-2 Mission Guide*, ESA, 2024. https://sentinel.esa.int/web/sentinel/missions/sentinel-2

[3] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019. https://arxiv.org/abs/1711.05101

[4] J. A. Riquelme, R. Tapia, and A. Garrido, "Land Cover Classification Using Sentinel-2 Imagery and Deep Learning," *Remote Sensing*, vol. 13, no. 1, 2021.

[5] S. Ramírez, *FastAPI Documentation*, 2024. https://fastapi.tiangolo.com/

[6] Plotly Technologies Inc., *Collaborative data science*, 2024. https://plotly.com